

LAPORAN TUGAS BESAR
MIKROPROSESOR DAN ANTARMUKA
WHACK A MOLE GAME DENGAN STM32F411CE6
IMPLEMENTASI WHACK A MOLE MENGGUNAKAN STM32



Kelompok: 4

1	Abdu Fattah Hidayatullah (101032300039)	[BAB 1, BAB 3, BAB 4]
2	Muhammad Aidil Fadhli A (101032300087)	[BAB 1, BAB 4]
3	Sandy Hidayatullah (101032300009)	[BAB 2, BAB 4]
4	Andreas Pangjiashun (101032300009)	[BAB 2, BAB 4]

Dosen: I Kadek Nuary Trisnawan, S.T., M.T.

UNIVERSITAS TELKOM
FAKULTAS TEKNIK ELEKTRO
PROGRAM STUDI TEKNIK KOMPUTER
SEMESTER 5 - TAHUN AKADEMIK 2025/2026

ABSTRAK

Laporan ini menjelaskan implementasi Whack A Mole menggunakan mikrokontroler STM32F411CEU6 untuk memenuhi CLO3. Sistem dirancang untuk permainan ketangkasan yang melatih reflek motorik pengguna melalui kombinasi input 9 tombol secara acak dengan representasi LED sebagai Mole dengan visual berbasis LCD 16x2. Implementasi meliputi: (1) komunikasi serial I2C dan SPI untuk menghubungkan LCD 16x2 dan LED (2) timer interrupt untuk waktu permainan terutama pada variable timeleft untuk perhitungan mundur dari game dan untuk mengatur interval nyala LED (3) external interrupt untuk button yang akan menerima input dari user.

Sistem terdiri dari 4 subsistem utama: Subsistem Input, Subsistem Timing, Subsistem Display dan Subsistem Display. Hasil pengujian menunjukkan bahwa Whack a Mole berjalan sesuai rencana, mampu menerima input, memberikan output, dan logika game berjalan lancar. Implementasi berbasis register menunjukkan pemahaman arsitektur STM32 pada level peripheral.

BAB I: PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi saat ini khususnya di dunia komputer dan *Embedded System* membuat manusia saat ini bergantung kepada mikrokontroller/mikroprosesor sebagai alat pemroses utama yang lebih efisien, kecil dan dapat diandalkan. Dalam konteks ini kami menggunakan Mikrokontroller STM32 dengan tipe STM32F411CEU6. Mikrokontroller ini penting karena menjadi pondasi utama di proyek ini sebelum, ditambahkan peripherals lainnya seperti LED, breadboard dan lainnya. Kami menggunakan STM32 tipe tersebut karena harganya lebih terjangkau, memenuhi kriteria minimum, dan selama pembelajaran kami dikelas juga menggunakan tipe tersebut. Di samping pemilihan mesin utama tersebut, perangkat juga dapat berjalan karena adanya implementasi pada protokol komunikasi serial, khususnya I2C dan SPI. Kedua protokol ini berfungsi sebagai jembatan pertukaran data antara mikrokontroler dengan peripherals lainnya, di mana SPI untuk kecepatan transmisi data lebih tinggi untuk pengendalian modul tampilan, sementara I2C menangani komponen pendukung lainnya dengan kecepatan rendah.

Nantinya kendali selanjutnya akan ditentukan oleh timer dan interrupt. Timer berfungsi untuk menghasilkan interval waktu secara acak untuk mengatur logika permainan, sedangkan Interrupt digunakan untuk merekam respons input tombol dari pemain yang nantinya akan menentukan respon komponen lainnya, Register disini berfungsi pada GPIO contohnya yang mengatur LED, LCD dan Button seperti Moder, ODR, IDR. Kami mengambail proyek Whack a Mole ini karena menurut kami cukup kompleks dan menarik.

1.2 Rumusan Masalah

1. Bagaimana mengimplementasikan protokol I2C berbasis HAL Library untuk mengontrol LCD 16x2 ?
2. Bagaimana mengkonfigurasi TIM2/TIM3 sebagai *timer interrupt* presisi 1 detik untuk *countdown* permainan?
3. Bagaimana merancang logika permainan *Whack-A-Mole* yang dapat menangani input 9 tombol dan output 9 LED secara simultan?
4. Bagaimana mengatasi masalah *bouncing* pada tombol mekanik menggunakan software?

1.3 Tujuan

- **Umum:** Proyek ini bertujuan untuk mengimplementasikan beragam fitur STM32 kedalam sebuah permainan sederhana bernama whack-a-mole, dimainkan dengan cara menekan tombol yang ditandai nyala LED, jika ditekan tepat waktu akan menambah poin skor, akan ada kenaikan kecepatan permainan setiap 10 skor.
- **Khusus:**
 - Serial Communication (20%): Mampu mengirim data ke LCD melalui protokol I2C menggunakan fungsi HAL_I2C_Master_Transmit.
 - Timer & Interrupt (40%): Mampu membangkitkan *update event* setiap 1ms atau 1s menggunakan *prescaler* dan *auto-reload register* untuk manajemen waktu *game*.
 - HAL Library Implementation (30%): Mampu mengimplementasikan driver periferan menggunakan pustaka STM32 HAL untuk efisiensi kode.
 - Integration (10%): Mengintegrasikan Keseluruhan sistem, timer, interrupt, dan sistem lainnya.

1.4 Batasan Masalah

- Hardware: STM32F411CEU6
- Software: Visual Studio Code
- Lingkup: Platform.IO

1.5 Manfaat

- Pembelajaran: Memberikan keuntungan belajar yang sangat lengkap karena tidak hanya fokus pada satu bagian pemrograman, tetapi juga memahami sistem tertanam sebagai keseluruhan yang saling berkaitan. Dengan mengatur lama permainan, waktu untuk setiap level, dan respons cepat saat menekan tombol, kami dilatih untuk memahami konsep sistem waktu nyata dan pemrograman berbasis peristiwa, di mana sistem perlu bereaksi tepat waktu terhadap kejadian seperti gangguan timer dan gangguan tombol. Selain itu, merancang alur permainan berdasarkan level melatih pola pikir sistematis melalui penggunaan mesin keadaan, yang merupakan konsep penting dalam pengembangan sistem tertanam pada skala industri.
- Teknis: Tugas ini bertujuan untuk meningkatkan pengetahuan tentang cara menggunakan berbagai perangkat tambahan pada mikrokontroler STM32F4 secara menyeluruh. Kami belajar menggunakan I2C untuk menghubungkan LCD, SPI untuk mengontrol LED, timer untuk mengatur waktu permainan dan kecepatan modul, interrupt untuk membaca tombol dengan cepat, serta PWM untuk menunjukkan tingkat dan memberikan suara atau tampilan umpan balik. Ini semua akan melatih mahasiswa dalam mengelola komunikasi data, menyinkronkan perangkat tambahan, dan memisahkan pekerjaan antara rutinitas layanan interupsi dan program utama. Selain itu, proses pelaksanaan dan pemecahan masalah juga membantu kami memahami tantangan seperti keterlambatan, waktu, dan stabilitas sistem, yang tidak bisa didapatkan hanya dari pembelajaran teori.

- Aplikatif: Konsep yang digunakan dalam pekerjaan ini sangat sesuai dengan penggunaan sistem embedded di kehidupan sehari-hari. Mekanisme batas waktu, penambahan kecepatan sistem sesuai level, serta penggunaan PWM sebagai tanda intensitas atau performa mirip dengan yang ada dalam industri, otomasi, dan IoT, seperti pengaturan mesin berdasar waktu, sistem alarm yang bisa menyesuaikan, dan kontrol perangkat berdasarkan peristiwa. Maka dari itu, proyek Whack-a-Mole ini tidak hanya berperan sebagai latihan belajar, tetapi juga menjadi dasar keterampilan praktis yang bisa digunakan dalam pengembangan sistem embedded, otomatisasi, dan teknologi pintar di masa depan.

BAB II: LANDASAN TEORI

2.1 Mikrokontroler STM32F411CEU6

STM32F4 merupakan seri MCU performa tinggi dengan instruksi FPU (Floating Point Unit) dan DSP (Digital Signal Processing). Di mana arsitektur intinya berbasis ARM Cortex-M4 32-bit RISC. Mikrokontroler ini mampu beroperasi pada frekuensi hingga 100 MHz, memberikan kecepatan eksekusi instruksi yang sangat memadai untuk aplikasi sistem tertanam real-time.

Seri STM32F411CEU6 atau yang sering dikenal dengan Black Pill dipilih dalam proyek ini karena memiliki spesifikasi yang mumpuni untuk menangani *multiple peripherals* secara bersamaan.

2.2 Serial Communication

2.2.1 Protokol I2C

- **Inisialisasi:** Fungsi `HAL_I2C_Init()` digunakan untuk mengonfigurasi parameter komunikasi seperti *clock speed* (Standard Mode 100kHz), *duty cycle*, dan alamat perangkat tanpa perlu menyentuh register CR1/CR2 secara manual.
- **Transmisi:** Fungsi `HAL_I2C_Master_Transmit()` menangani seluruh proses pengiriman data ke LCD, mulai dari membangkitkan kondisi *Start*, mengirim alamat *slave*, mengirim data *payload*, menunggu sinyal ACK, hingga membangkitkan kondisi *Stop*.

2.2.2 Protokol SPI

- **Inisialisasi:** Fungsi `HAL_SPI_Init()` digunakan untuk mengatur parameter komunikasi seperti mode (Master/Slave), prescaler *baud rate*, polaritas *clock* (CPOL), dan fase *clock* (CPHA) sesuai struktur `SPI_HandleTypeDef`.
- **Transmisi Data:** Fungsi `HAL_SPI_Transmit()` digunakan untuk mengirimkan data *buffer* (pola nyala LED) dari mikrokontroler ke perangkat *slave*. Karena *Shift Register* hanya menerima data (tidak mengirim balik), fungsi *Receive* tidak digunakan dalam implementasi ini.

- **Kontrol Chip Select:** Pin CS/SS dikendalikan secara manual menggunakan HAL_GPIO_WritePin() untuk memberi sinyal *latch* data ke *Shift Register*.

2.3 Timer dan Counter

- **Base Timer:** Fungsi HAL_TIM_Base_Init() menginisialisasi timer dasar.
- **Interrupt:** Untuk mengaktifkan timer dengan interupsi, digunakan fungsi HAL_TIM_Base_Start_IT().
- **Callback:** HAL_TIM_PeriodElapsedCallback() akan dipanggil otomatis setiap kali timer mencapai batas hitungan (overflow).

2.4 Interrupt System

- **Konfigurasi Prioritas:** Fungsi HAL_NVIC_SetPriority() dan HAL_NVIC_EnableIRQ() digunakan untuk mengaktifkan jalur interupsi tertentu dan mengatur tingkat prioritasnya (misalnya, prioritas Timer lebih tinggi dari Tombol).
- **Penanganan Interupsi (Handler):** Fungsi HAL_GPIO_EXTI_IRQHandler() dipanggil secara otomatis di dalam fungsi utama ISR (*Interrupt Service Routine*). Fungsi ini bertugas membersihkan *interrupt flag* secara otomatis.
- **Callback:** Logika program pengguna (seperti menambah skor saat tombol ditekan) diletakkan di dalam fungsi HAL_GPIO_EXTI_Callback(). Fungsi ini merupakan fitur HAL yang memungkinkan pemisahan logika program dari penanganan register interupsi tingkat rendah.

2.5 GPIO Configuration

- **Write:** `HAL_GPIO_WritePin(GPIOx, GPIO_Pin, PinState)` untuk mengubah status pin menjadi High/Low.
- **Read:** `HAL_GPIO_ReadPin(GPIOx, GPIO_Pin)` untuk membaca status input digital.
- **Toggle:** `HAL_GPIO_TogglePin(GPIOx, GPIO_Pin)` untuk membalik status pin.

BAB III: REQUIREMENT DAN SPESIFIKASI SISTEM

3.1 System Requirements

Sistem *Whack-A-Mole* bertujuan untuk melatih ketangkasan respons dan koordinasi motorik pemain melalui permainan interaktif berbasis mikrokontroler. Persyaratan utama:

- **Input Handling:** Sistem harus mampu menangani input dari 10 tombol tactile (1 tombol Start dan 9 tombol permainan) menggunakan konfigurasi Internal Pull-Up dengan waktu respons deteksi kurang dari 20ms (termasuk software debouncing).
- **Output/Display:** Sistem menampilkan informasi status (Skor, Waktu & Level) pada LCD 16x2 dengan laju pembaruan 1 Hz / Detik atau seketika saat skor berubah. Serta mengontrol 9 LED indikator permainan secara acak tanpa *flicker*.
- **Communication:** Mikrokontroler menggunakan protokol I2C untuk mengirim data karakter ke LCD, dan protokol SPI untuk mengirim data pola nyala LED ke IC Shift Register (74HC595) secara serial.

3.2 Functional Requirements

Tabel 1 Functional Requirements

ID	Requirement	Priority
FR-01	Sistem dapat mengacak nyala 1 dari 9 LED setiap interval waktu tertentu.	High
FR-02	Sistem dapat mendeteksi input tombol yang sesuai dengan LED yang menyala dan menambah skor (+1).	High
FR-03	Sistem dapat mengurangi waktu permainan (countdown) setiap 1 detik.	High
FR-04	LCD menampilkan Skor, Sisa Waktu dan Level.	Medium
FR-05	Permainan berhenti dan menampilkan "Game Over" saat waktu habis.	Medium

3.3 System Specifications

3.3.1 Hardware Specifications

Tabel 2 Komponen Utama

Komponen	Spesifikasi	Jumlah	Fungsi
Mikrokontroler	STM32F411CEU6	1	Processing unit
LCD	16x2 I2C	1	Display output
Button	Push button	10	User input
LED	5mm	9	Visual indicator
Buzzer	Mini	1	Audio Indicator

Tabel 3 Pin Mapping

Pin STM32	Connect To	Fungsi	Peripheral
PB6	LCD SCL	I2C Clock	I2C1_SCL
PB7	LCD SDA	I2C Data	I2C1_SDA
PA5	Shift Reg SCK	SPI Clock	SPI1_SCK
PA7	Shift Reg DATA	SPI MOSI	SPI1_MOSI
PA4	Shift Reg LATCH	Control Signal	GPIO_Output
PA0	Button 1	Input Pemain	GPIO_Input (Pull-Up)
PA1	Button 2	Input Pemain	GPIO_Input (Pull-Up)
PA2	Button 3	Input Pemain	GPIO_Input (Pull-Up)
PA3	Button 4	Input Pemain	GPIO_Input (Pull-Up)

3.3.2

Pin STM32	Connect To	Fungsi	Peripheral
PB6	LCD SCL	I2C Clock	I2C1_SCL
PA6	Button 5	Input Pemain	GPIO_Input (Pull-Up)
PA8	Button 6	Input Pemain	GPIO_Input (Pull-Up)
PB0	Button 7	Input Pemain	GPIO_Input (Pull-Up)
PB1	Button 8	Input Pemain	GPIO_Input (Pull-Up)
PB2	Button 9	Input Pemain	GPIO_Input (Pull-Up)
PB12	Button 10	Start	GPIO_Input (Pull-Up)
PB10	Buzzer	Audio Output	GPIO_Output

Software

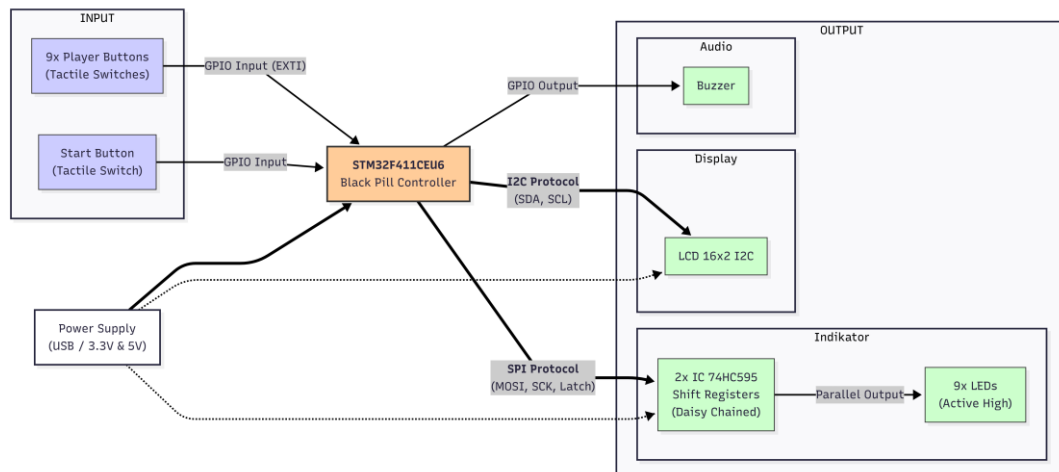
Specifications

- Development Tool: VSCode + PlatformIO
- Programming: HAL Library, register-based
- Clock: 16 Mhz (HSI) atau 100MHz (HSE + PLL)
- Communication: I2C (100kHz) / UART (9600 baud)
- Timer: 1MHz (1 μ s resolution)

BAB IV: PERANCANGAN SISTEM

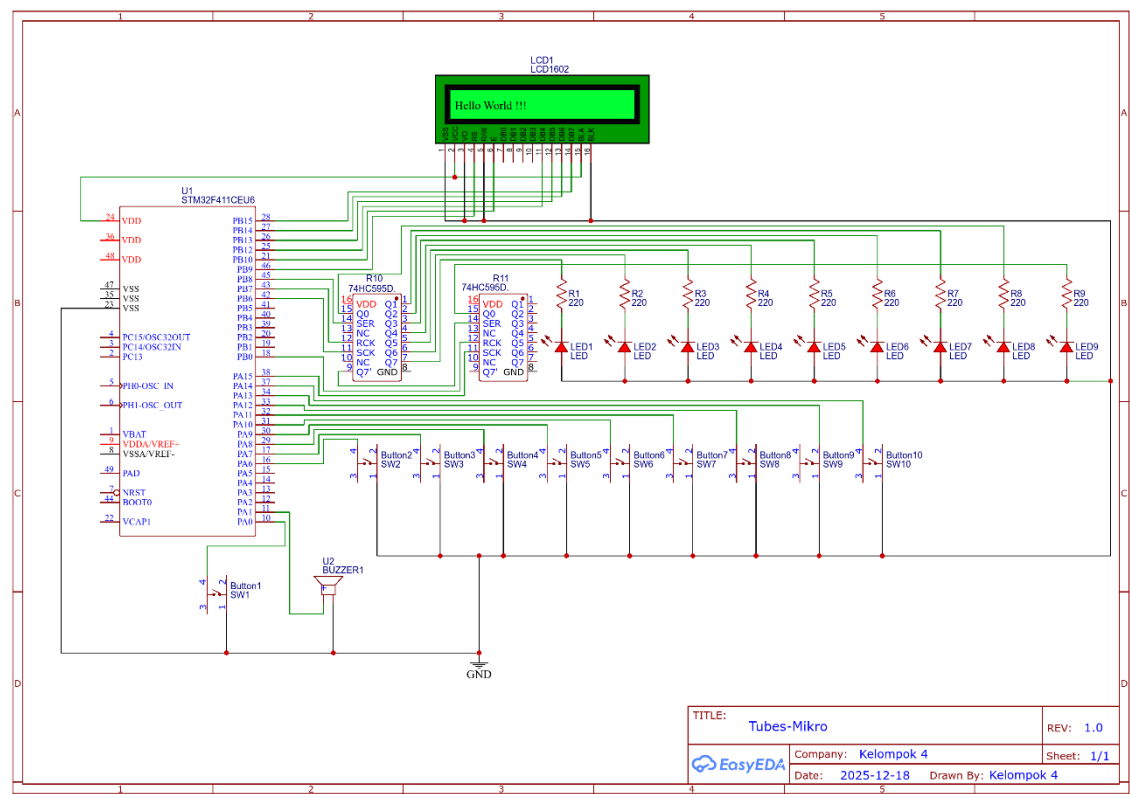
4.1 Arsitektur Sistem

Block Diagram:



4.2 Desain Hardware

4.2.1 Schematic Diagram



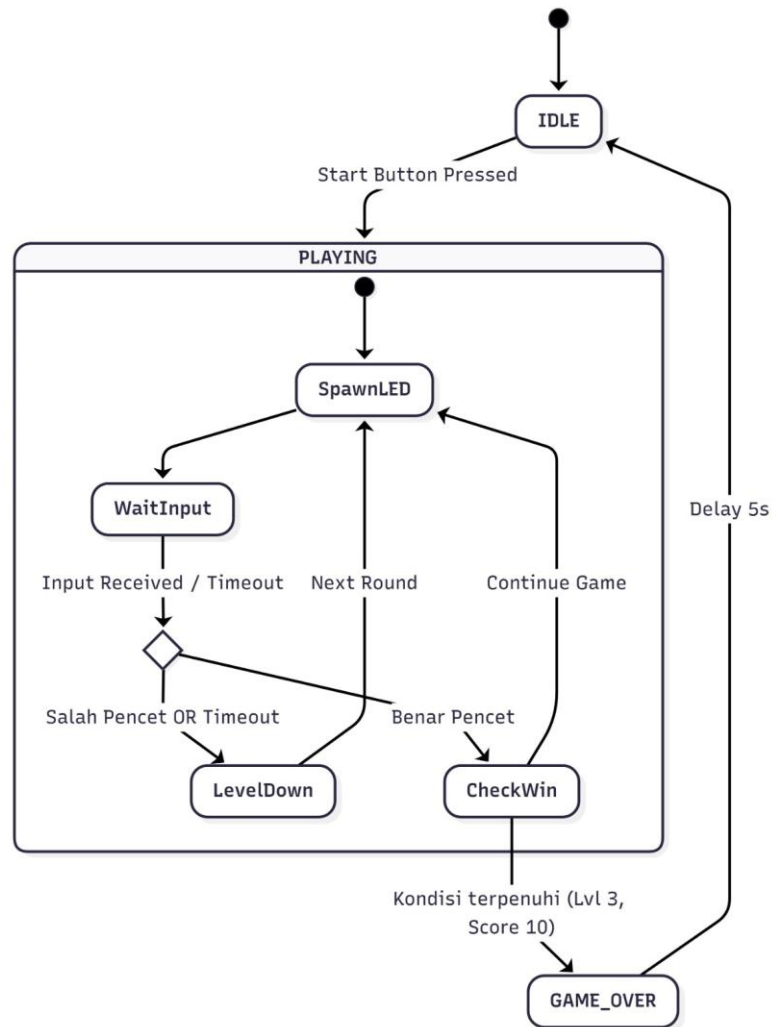
4.2.2 Bill of Materials (BOM)

No	Komponen	Nilai/Tipe	Qty	Harga (Rp)
1	STM32 BLACK PILL	F411CEU6	1	80000
2	LCD 12C	16x2	1	25.000
3	LED	5mm	9	1000
4	Push Button	4P	9	2000
5	Kabel Jumper	40pcs	1	16000
6	Breadboard	Long	2	15000
7	IC 74HC595	DIP 16P	2	2000

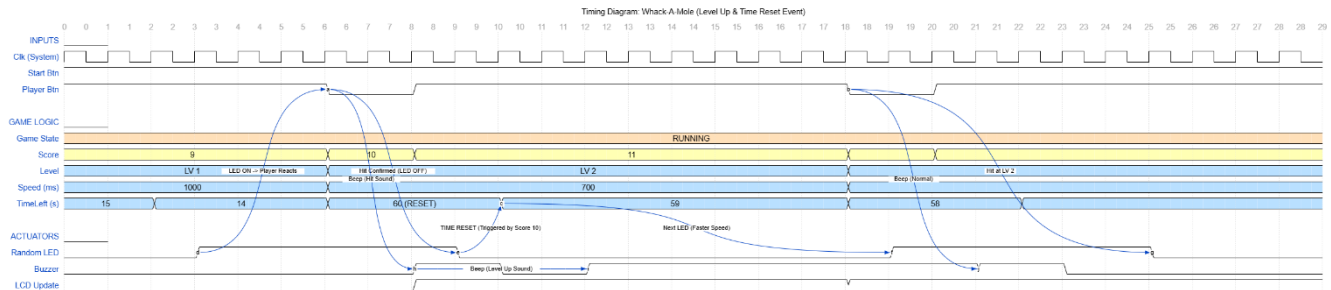
Link Flowchart: [Flowchart](#)

https://drive.google.com/drive/folders/1F_8U8mp1X8mU6693oXQiEAilnri3zhau?usp=drive_link

4.3.2 State Machine Diagram



4.3.3 Timing Diagram



4.4 Desain Subsystem

4.4.1 Subsystem Input (EXTI & GPIO)

Design Debouncing:

Waktu stabil button setelah bounce: 20 ms

State Machine:

States:

Transitions:

- Released → Debouncing: Transisi terjadi seketika saat *Interrupt* (EXTI) mendeteksi tepi turun (*Falling Edge*) pada pin tombol.
- Debouncing → Pressed: Sistem memulai timer software selama 20ms. Jika setelah 20ms logika pin masih LOW, maka status dianggap valid dan aksi game dijalankan.
- Pressed → Released: Jika logika pin kembali HIGH (tombol dilepas).

4.4.2 Subsystem Timing (Timer Dynamics)

Konfigurasi Timer Counter:

- Clock source: $APB1 \times 1 = 16\text{MHz}$
- Prescaler: $PSC = 16 - 1 = 15$
- $f_{\text{timer}} = 16\text{MHz} / 16 = 1\text{MHz}$
- Resolution: $1\mu\text{s}$ per count

Auto-Reload Register (ARR):

Untuk interrupt setiap 1ms:

$$ARR = (1\text{ms} \times 1\text{MHz}) - 1 = 999$$

4.4.3 Subsystem Display (I2C Communication)

I2C Clock Calculation:

I2C Clock Calculation:

$$CCR = f_APB1 / (2 \times f_I2C)$$

Target Frekuensi : 100 kHz

Clock Input : 16 MHz

Perhitungan CCR (Clock Control Register):

$$CCR = \frac{16000000}{2 \cdot 100000} = 80$$

80 Desimal / 0x50 HexState Machine:

States:

Transitions:

- Idle → Start: Master (STM32) menarik SDA ke Low saat SCL High.
- Start → Address: Master mengirimkan alamat LCD (0x27 atau 0x3F) + bit Write.
- Address → Write_Data: Jika Slave (LCD) mengirimkan bit ACK (Low).
- Write_Data → Stop: Setelah data karakter/perintah selesai dikirim, Master melepas SDA ke High saat SCL High.

4.4.4 Subsistem Output (SPI & Shift Register)

Konfigurasi SPI:

- **Mode:** Master Transmit Only (Simplex).
- **Data Size:** 16-bit (Kontrol 2 buah IC 74HC595 diserikan).
 - **IC 1 (*Lower Byte*):** Mengontrol LED 1-8.
 - **IC 2 (*Upper Byte*):** Mengontrol LED 9.
- **Baud Rate Prescaler:** $f_{PCLK} / 32$.

Alur Pengiriman Data (Flow):

- **Latch Low:** Pin Latch (PA4) ditarik ke logika LOW untuk membuka buffer penyimpanan.
 - **Transmit:** STM32 mengirimkan 16-bit data pola LED melalui jalur MOSI sinkron dengan SCK.
 - **Latch High:** Pin Latch (PA4) ditarik ke logika HIGH. Pada momen *rising edge* ini, IC 74HC595 akan memperbarui output fisiknya secara serentak, sehingga LED menyala tanpa *flicker*.
-

BAB V: IMPLEMENTASI

5.1 Implementasi Hardware

https://drive.google.com/drive/folders/1mraAOwRyifNUjTe7elch8JBc6c7jjrF6?usp=drive_link

5.2 Implementasi Software (Register-Based)

5.2.1 RCC Configuration

Kode:

```
RCC->AHB1ENR |= (1<<0) | (1<<1); //Mengaktifkan GPIOA GPIOB
```

```
RCC->APB1ENR |= (1<<0); // Mengaktifkan TIM2 (Buzzer)
```

```
RCC->APB2ENR |= (1<<14); // Mengaktifkan SYSCFG (kontrol interrupt)
```

Penjelasan Register:

No	Register	Bit	Value	Fungsi
1	RCC->AHB1ENR	0	1	Enable GPIOA
2	RCC->AHB1ENR	1	1	Enable GPIOB
3	RCC->APB1ENR	0	1	Enable TIM2
4	RCC->APBENR	14	1	Enable Konfigurasi Kontrol sistem

5.2.2 GPIO Configuration

Kode:

```
void init_ShiftRegister(void) {  
    // Mengaktifkan Clock Port A  
    RCC->AHB1ENR |= (1 << 0);  
  
    // Mengubah Mode PA4 PA5 PA7 sebagai OUTPUT  
    GPIOA->MODER |= (1<<(4*2)) | (1<<(5*2)) | (1<<(7*2));  
  
    // Pengatur Kecepatan HIGH SPEED kode '10'  
    GPIOA->OSPEEDR |= (2<<(4*2)) | (2<<(5*2)) | (2<<(7*2));  
  
}  
  
void init_LCD_I2C(void) {
```

```

    // Mengaktifkan Clock untuk Port B
    RCC->AHB1ENR |= (1 << 1);

    // Mengatur pin PB6(SCL) PB7(SDA) sebagai output
    GPIOB->MODER |= (1<<(6*2)) | (1<<(7*2));

    // Mengubah Tipe Output menjadi OPEN-DRAIN Kode '1'
    GPIOB->OTYPER |= (1<<6) | (1<<7);

    // Mengatur Kecepatan menjadi VERY HIGH SPEED Kode '11'
    GPIOB->OSPEEDR |= (3<<(6*2)) | (3<<(7*2));
}

void init_Buzzer_PWM(void) {
    // Mengaktifkan Clock Port B & Timer 2
    RCC->AHB1ENR |= (1 << 1); // GPIO Port B
    RCC->APB1ENR |= (1 << 0); // TIM2

    // Mengubah Mode PB10 sebagai ALTERNATE FUNCTION
    GPIOB->MODER |= (2 << (10 * 2));

    // Register AFR[1] (High Register) untuk pin 8-15
    GPIOB->AFR[1] |= (1 << 8);
}

void init_Input_PortA(void) {
    // Mengaktifkan Clock Port A
    RCC->AHB1ENR |= (1 << 0);

    // Reset mode input PA0, PA1, PA2, PA3, PA6, PA
    GPIOA->MODER &= ~((3<<(0*2)) | (3<<(1*2)) | (3<<(2*2)) |
(3<<(3*2)) | (3<<(6*2)) | (3<<(8*2))));

    // Mengaktifkan Resistor PULL-UP Internal
    GPIOA->PUPDR |= (1<<(0*2)) | (1<<(1*2)) | (1<<(2*2)) |
(1<<(3*2)) | (1<<(6*2)) | (1<<(8*2)));
}

void init_Input_PortB(void) {
    // 1. Mengaktifkan Clock Port B
    RCC->AHB1ENR |= (1 << 1);
    GPIOB->MODER &= ~((3<<(12*2)) | (3<<(13*2)) | (3<<(14*2)) | (3<<(15*2)));
    GPIOB->PUPDR |= (1<<(12*2)) | (1<<(13*2)) | (1<<(14*2)) |
(1<<(15*2)));
}

```

Penjelasan Register:

Register	Bit	Value	Nama Bit / Fungsi Dasar	Deskripsi
RCC->AHB1ENR	Bit 0	1	GPIOAEN	Mengaktifkan arus listrik/clock ke Port A. Wajib dinyalakan agar Shift Register (PA4, PA5, PA7) dan Tombol (PA0-PA8) bisa bekerja.
RCC->AHB1ENR	Bit 1	1	GPIOBEN	Mengaktifkan arus listrik/clock ke Port B. Digunakan untuk LCD (PB6-PB7), Buzzer (PB10), dan Tombol (PB12-PB15).
RCC->APB1ENR	Bit 0	1	TIM2EN	Mengaktifkan clock untuk periferan Timer 2. Diperlukan agar sinyal PWM untuk Buzzer bisa dibangkitkan.
GPIOx->MODER	PA4, PA5, PA7	01	General Purpose Output	Mengatur pin Shift Register (Latch, Clock, Data) sebagai Output agar CPU bisa mengirim sinyal data ke IC 74HC595.

GPIOx->MODER	PB6, PB7	01	General Purpose Output	Mengatur pin LCD (SCL, SDA) sebagai Output manual (<i>Bit-Banging</i>), bukan mode Alternate Function I2C hardware.
GPIOx->MODER	PA0-3, PA6, PA8 PB12-15	00	Input Mode	Mengatur pin Tombol agar berfungsi sebagai Input untuk membaca tegangan masuk (ditekan/tidak).
GPIOx->MODER	PB10	10	Alternate Function	Mengalihkan kendali pin PB10 dari GPIO biasa ke periferal internal (Timer 2) untuk output sinyal PWM Buzzer.
GPIOx->OTYPER	PB6, PB7	1	Output Open-Drain	Mengatur tipe output LCD menjadi Open-Drain. Sangat krusial untuk protokol I2C agar tidak terjadi korsleting (<i>short</i>) antar jalur data.
GPIOx->OSPEEDR	PA4, PA5, PA7	10	High Speed	Meningkatkan kecepatan transisi sinyal (Slew Rate) pada Shift Register agar data serial terkirim cepat dan tajam.

GPIOx->OSPEEDR	PB6, PB7	11	Very High Speed	Mengatur kecepatan pin LCD ke maksimum agar komunikasi I2C Bit-Banging berjalan stabil tanpa <i>lag</i> .
GPIOx->PUPDR	PA0-3, PA6, PA8 PB12-15	01	Pull-Up Resistor	Mengaktifkan resistor internal ke VCC (3.3V). Menjaga sinyal tombol tetap HIGH (1) saat tidak ditekan (Active Low Logic).
GPIOB->AFR[1]	Pin 10 (Bit 8-11)	0001	AF1 (TIM2)	Memetakan (Multiplexing) fungsi Timer 2 Channel 3 ke kaki fisik PB10.

5.2.3 I2C Configuration

Kode:

```
// Definisi Kontrol Pin SDA (PB7)
#define SDA_HIGH (GPIOB->ODR |= (1 << 7)) // Set PB7 High
#define SDA_LOW (GPIOB->ODR &= ~(1 << 7)) // Set PB7 Low

// Definisi Kontrol Pin SCL (PB6)
#define SCL_HIGH (GPIOB->ODR |= (1 << 6)) // Set PB6 High
#define SCL_LOW (GPIOB->ODR &= ~(1 << 6)) // Set PB6 Low
```

Penjelasan Register:

Register	Bit	Operasi	Deskripsi Teknis
GPIOB->ODR	Bit 7	**Set (	SDA_HIGH
GPIOB->ODR	Bit 7	Clear (&= ~1)	Output Data Register Low.

			Menulis logika '0' ke bit 7. Transistor internal menyala (ON) dan menghubungkan pin PB7 langsung ke Ground (0V). menciptakan sinyal Logic 0.
GPIOB->ODR	Bit 6	**Set (	SCL_HIGH
GPIOB->ODR	Bit 6	Clear (&= ~1)	Output Data Register Low. Menulis logika '0' ke bit 6. Memaksa pin PB6 ke Ground. Digunakan untuk menahan clock atau persiapan pengiriman bit data berikutnya.

5.2.4 Timer Configuration

Kode:

```
void Timer_Config(void) {
    // Mengaktifkan Clock untuk TIM2 (APB1 Bus)
    RCC->APB1ENR |= (1 << 0);

    // Mengatur Prescaler (PSC) CPU Clock (16MHz) dibagi 16 = 1 MHz
    Timer Clock
    TIM2->PSC = 16 - 1;

    // Mengatur Auto-Reload Register (ARR) Menentukan Periode. Misal
    1000 tick = 1ms = 1kHz
    TIM2->ARR = 1000 - 1;

    // Mengatur Capture/Compare Mode (CCMR2) untuk Channel 3 (6 <<
    4): PWM Mode 1 (1 << 3): Output Compare Preload Enable
    TIM2->CCMR2 = (6 << 4) | (1 << 3);

    // Mengaktifkan Output Channel 3 (CCER)
    TIM2->CCER |= (1 << 8);

    // 6. Menyalakan Counter (CR1)
    // Bit 0 (CEN) diset High agar Timer mulai bekerja
    TIM2->CR1 |= (1 << 0);

    // 7. Set Duty Cycle Awal 0 (Suara Mati)
    TIM2->CCR3 = 0;
}
```

Penjelasan Register:

Register	Bit / Value	Nama Parameter	Deskripsi Fungsi dalam Proyek
RCC->APB1ENR	Bit 0 (Set 1)	TIM2EN	Timer 2 Clock Enable. Menyalakan sumber daya (clock) untuk modul Timer 2. Tanpa ini, register timer tidak bisa diakses.
TIM2->PSC	15 (16 - 1)	Prescaler	Pembagi Frekuensi Clock. Membagi clock CPU (16 MHz) dengan 16. Hasilnya: Timer berjalan dengan kecepatan 1 MHz (1 tick = 1 mikrodetik). mempermudah perhitungan nada audio.
TIM2->ARR	999 (1000 - 1)	Auto-Reload Register	Pengatur Frekuensi (Nada). Menentukan batas atas hitungan timer (Periode). Ketika counter mencapai angka ini, timer akan reset ke 0. Nilai inilah yang diubah-ubah untuk menghasilkan nada.
TIM2->CCMR2	Bit 4-6 (Val: 6)	OC3M (PWM Mode 1)	Output Compare Mode. Mengatur Channel 3 ke mode PWM 1. Logikanya: Selama hitungan counter lebih kecil dari CCR3, output = HIGH. Sisanya = LOW.

TIM2->CCMR2	Bit 3 (Set 1)	OC3PE	Preload Enable. Memastikan perubahan sinyal PWM berjalan mulus saat nilai register diupdate di tengah-tengah lagu.
TIM2->CCER	Bit 8 (Set 1)	CC3E	Capture/Compare 3 Output Enable. Menghubungkan logika internal Timer Channel 3 ke kaki pin fisik PB10. Tanpa bit ini, timer berjalan tapi suara tidak keluar.
TIM2->CR1	Bit 0 (Set 1)	CEN	Counter Enable. Instruksi "Start". Memerintahkan Timer untuk mulai menghitung dari 0.
TIM2->CCR3	0 (Dinamis)	Capture/Compare Register	Pengatur Duty Cycle (Volume). Menentukan lebar pulsa positif. Dalam konteks buzzer: • Nilai 0 = Suara Mati. • Nilai 50% dari ARR = Suara Paling Nyaring .

5.2.5 External Interrupt Configuration

Kode:

```
void Input_Init(void) {
    // Mengaktifkan Clock SYSCFG (System Configuration)
    RCC->APB2ENR |= (1 << 14);

    // Routing Pin ke Jalur Interrupt (SYSCFG_EXTICR)
```

```

// Konfigurasi Port A (PA0-PA3) -> EXTI0-EXTI3
SYSCFG->EXTICR[0] &= ~0xFFFF;

// Konfigurasi Port A (PA6) -> EXTI6
SYSCFG->EXTICR[1] &= ~(0xF << 8);

// Konfigurasi Port B (PB12-PB15) -> EXTI12-EXTI15
SYSCFG->EXTICR[3] = 0x1111;

// Buka gerbang interrupt
EXTI->IMR |= 0xF04F;

// Falling Edge Trigger . Deteksi tombol ditekan (High -> Low)
EXTI->FTSR |= 0xF04F;

// Mendaftarkan fungsi handler ke Core CPU agar bisa dieksekusi
NVIC_EnableIRQ(EXTI0_IRQn);      // Tombol 1
NVIC_EnableIRQ(EXTI1_IRQn);      // Tombol 2
NVIC_EnableIRQ(EXTI2_IRQn);      // Tombol 3
NVIC_EnableIRQ(EXTI3_IRQn);      // Tombol 4
NVIC_EnableIRQ(EXTI9_5_IRQn);     // Tombol 5
NVIC_EnableIRQ(EXTI15_10_IRQn);  // Tombol 6-9
}

```

Penjelasan Register:

Register	Bit	Value (Nilai)	Nama Bit	Fungsi Spesifik dalam Proyek
RCC->APB2ENR	14	1	SYSCFGEN	Enable System Configuration Clock. Menyalakan modul SYSCFG.
SYSCFG->EXTICR[0]	0-15	0000	EXTI0 - EXTI3	Select Port A for Line 0-3. Mengarahkan jalur interrupt EXTI0 s/d EXTI3 agar membaca sinyal dari PA0, PA1, PA2, PA3.

				(Nilai 0 = Port A).
SYSCFG->EXTICR[1]	8-11	0000	EXTI6	Select Port A for Line 6. Mengarahkan jalur interrupt EXTI6 agar membaca sinyal dari PA6.
SYSCFG->EXTICR[3]	0-15	0x1111	EXTI12 - EXTI15	Select Port B for Line 12-15. Mengisi 4 nibble dengan nilai 0001 (Port B). Ini mengarahkan EXTI12 s/d EXTI15 agar membaca sinyal dari PB12, PB13, PB14, PB15.
EXTI->IMR	0-3, 6, 12-15	1	MRx (Mask)	Interrupt Unmask (Enable). Membuka interrupt untuk line 0, 1, 2, 3, 6, 12, 13, 14, 15. Jika bit ini 0, sinyal tombol akan diabaikan oleh interrupt controller.
EXTI->FTSR	0-3, 6, 12-15	1	TRx (Falling)	Falling Trigger Selection. Mengatur agar interrupt terjadi saat tegangan turun. Ini sesuai

				dengan skema tombol Active Low (Pull-Up).
EXTI->RTSR	Semua	0	TRx (Rising)	Disable Rising Trigger. Memastikan interrupt TIDAK terjadi saat tombol dilepas, agar tidak terjadi double input.
NVIC->ISER[x]	Terkait	1	SETENA	Interrupt Set-Enable. Diakses via fungsi NVIC_EnableIRQ(). Memberi izin kepada CPU Cortex-M4 untuk menghentikan program utama dan melompat ke fungsi IRQHandler saat tombol ditekan.

5.2.6 Interrupt Service Routines

Kode:

```
// Handler untuk Tombol 1 (PA0) - Jalur EXTI0
void EXTI0_IRQHandler(void) {
    if(EXTI->PR & (1<<0)) {    // Cek apakah EXTI0 yang mentrigger?
        Btn_Check(0);          // Jalankan logika tombol (ID: 0)
        EXTI->PR |= (1<<0);    // Clear Pending Flag
    }
}

// ... (EXTI1, EXTI2, EXTI3 sama)

// Handler Gabungan untuk Tombol 6-9 (PB12-PB15) - Jalur EXTI10-15
void EXTI15_10_IRQHandler(void) {
```

```

// Cek bit spesifik untuk mengetahui tombol mana yang ditekan
if(EXTI->PR & (1<<12)) { Btn_Check(5); EXTI->PR |= (1<<12); } //
Tombol 6
if(EXTI->PR & (1<<13)) { Btn_Check(6); EXTI->PR |= (1<<13); } //
Tombol 7
if(EXTI->PR & (1<<14)) { Btn_Check(7); EXTI->PR |= (1<<14); } //
Tombol 8
if(EXTI->PR & (1<<15)) { Btn_Check(8); EXTI->PR |= (1<<15); } //
Tombol 9
}
void Btn_Check(int id) {
// Cek apakah selisih waktu dari interrupt terakhir > 150ms
if ((Tick - LastInt) > 150) {
    Input = id;        // Update variabel global Input
    LastInt = Tick;    // Simpan waktu penekanan terakhir
}
}

```

Langkah	Proses	Penjelasan Teknis
Checking	if(EXTI->PR & (1<<x))	Safety Check. Memastikan bahwa fungsi ini dipanggil oleh pin yang benar.
Processing	Btn_Check(id)	Logic & Debounce. CPU membandingkan waktu saat ini (Tick) dengan waktu interrupt sebelumnya. Jika intervalnya terlalu cepat (<150ms), interrupt dianggap noise dan diabaikan. Jika valid, variabel global Input diubah.
Clearing	EXTI->PR	= (1<<x) `

Analisis ISR:

- Execution time: waktu eksekusi singkat tanpa delay
- Priority: Level 0, supaya tidak delay dan respon cepat
- Critical section: supaya input dibaca valid

5.3 Pembagian Tugas Kelompok

Anggota	Tugas	Status
Semuanya	Hardware design & wiring	✓
Abdu Fattah H	SPI implementation	✓

Sandy Hidayatullah	I2C implementation	✓
Muhammad Aidil Fadhli A	Timer & interrupt	✓
Semuanya	Integration & testing	✓
Muhammad Aidil Fadhli A	Documentation	✓

BAB VI: PENGUJIAN DAN ANALISIS

6.1 Metodologi Pengujian

- Unit Testing: Pengujian Short (Multimeter), Pengujian 74HC595, Pengujian Input, Pengujian Output.
- Integration Testing: Pengujian Sinkronisasi Input dan Output, Pengujian Data skor ke LCD, Pengujian Buzzer apabila kena dan tidak kena.
- System Testing: Pengujian Alur permainan, Pengujian logika level, Pengujian reset skor, Pengujian penambahan skor dan level.
- Tools: Multimeter dan Obeng

6.2 Pengujian Sistem Keseluruhan

Skenario Pengujian:

1. Pengujian Inisialisasi Sistem
2. Pengujian Input Button & Interrupt
3. Logika Game
4. Timer
5. Output LED, LCD, dan Buzzer

Hasil Pengujian:

Pengujian	Kondisi	Hasil	Status
Inisiasi Sistem	Menghubungkan STM32 ke power/listrik.	LCD tampil	Berhasil
Pengujian Input	Menekan tombol input saat	Sistem mendeteksi ada tombol ditekan	Berhasil

	permainan secara acak sesuai mole yang keluar.		
Logika Game	Pemain mencapai skor kelipatan 10. Dan jika Pemain salah menekan tombol.	Level bertambah dan durasi permainan kembali penuh. Dan Skor kembali ke 0	Berhasil
Timer hitung mundur	Jika pemain membiarkan tanpa adanya input	Waktu tetap berkurang sampai 0	Berhasil
Output LED, LCD, dan Buzzer	Input dari pemain	LED menyala, LCD menyala dan buzzer berbunyi	Berhasil

Dokumentasi:

[\[Foto/video demonstrasi sistem berfungsi\]](#)

6.6 Troubleshooting

Problem	Cause	Solution
HAL Lib Err	Terlalu sensitif.	Ganti CMSIS.
I2C	Cairan liquid crystal display sudah pecah.	Ganti I2C.
Push button	Karena push button mengalami short atau tidak bisa di tekan.	Memindahkan button port dari sebelumnya PB8 ke PA0.

BAB VII: KESIMPULAN DAN SARAN

7.1 Kesimpulan

1. Implementasi I2C dan SPI pada part LCD terbukti efektif karena dapat meminimalisir penggunaan GPIO.

2. Penggunaan Timer dan Interrupt untuk menciptakan Input berhasil dilakukan dan terbukti menciptakan respon kecepatan yang tepat.
3. Penggunaan Register-level programming untuk membuat penggunaan memori lebih efisien berhasil dilakukan dan lebih baik daripada menggunakan HAL
4. Hasil pengujian secara fungsional sistem permainan berjalan sesuai rencana dan flowchart, fitur seperti kenaikan level otomatis, reset tiem, pengurangan skor dan lainnya dapat berjalan secara sinkron begitu juga pada tampilan LCD dan Audio.

7.2 Saran

1. Push button tidak sama dengan port lainnya agar tidak mengalami short atau conflict.
2. Meningkatkan pengetahuan tentang clock timer dan interupt
3. Penambahan level selanjutnya dan memperbagus tampilan

AI Editing Statement

Saya menyatakan bahwa saya menggunakan alat kecerdasan buatan (AI) hanya untuk keperluan editing, termasuk perbaikan tata bahasa, penyusunan kalimat, konsistensi istilah, dan peningkatan keterbacaan dokumen ini.

Seluruh ide, analisis, perhitungan, desain, dan konten substantif merupakan hasil pekerjaan saya sendiri. AI tidak digunakan untuk menghasilkan konten inti, menyelesaikan tugas teknis, atau membuat analisis secara otomatis.

Sebagai bentuk transparansi, berikut adalah daftar penggunaan AI beserta nama alat AI yang digunakan:

No.	Bagian yang Diedit	Jenis Bantuan AI	Nama AI yang Digunakan
1	Codingan (PIN Button)	Troubleshooting	Google Gemini 3.0 PRO
2	Proposal	Mencari Referensi	Google Gemini 3.0 PRO
3	Codingan (HAL>CMSIS)	Troubleshooting	Google Gemini 3.0 PRO

Nama Mahasiswa:

- Abdu Fattah Hidayatullah
- Muhammad Aidil Fadhli Awalludin
- Sandy Hidayatullah
- Andreas Pangjiashun

NIM:

- 101032300039

- 101032300087
- 101032300009
- 101032300002

Tanggal:
5 January 2026

DAFTAR PUSTAKA

- [1] STMicroelectronics. (2017). *RM0383 Reference Manual: STM32F411xC/xE Advanced Arm®-based 32-bit MCUs*. STMicroelectronics.
- [2] NXP Semiconductors. (2021). *UM10204: I2C-bus Specification and User Manual (Rev. 7.0)*. NXP Semiconductors.
- [3] Motorola Inc. (2004). *SPI Block Guide V04.01*. Motorola Intelligence and Information Systems Group.

LAMPIRAN

LAMPIRAN A: Full Source Code

```
#include "stm32f4xx.h"

#include <stdlib.h>

#define I2C_ADDR 0x27


volatile int Input = -1;

volatile uint32_t Tick = 0;

uint32_t Speed = 1000;

uint32_t Seed = 0;


void SysTick_Handler(void) {

    Tick++;

}


void Delay_us(uint32_t us) {

    uint32_t i;

    for(i=0; i<us*16; i++) { __NOP(); }

}


void Delay_ms(uint32_t ms) {

    uint32_t end = Tick + ms;

    while (Tick < end);

}
```

```

}

uint32_t GetRandom(void) {

    if (Seed == 0) Seed = Tick + 12345;

    Seed = Seed * 1664525 + 1013904223;

    return Seed;

}

/* = AUDIO BUZZER = */

void Audio_Init(void) {

    GPIOB->MODER &= ~(3 << (10 * 2));

    GPIOB->MODER |= (2 << (10 * 2));

    GPIOB->AFR[1] &= ~(0xF << 8);

    GPIOB->AFR[1] |= (0x1 << 8);


    TIM2->PSC = 100 - 1;

    TIM2->ARR = 1000 - 1;

    TIM2->CCR3 = 0;

    TIM2->CCMR2 = (6 << 4) | (1 << 3);

    TIM2->CCER |= (1 << 8);

    TIM2->CR1 |= (1 << 0);

}

```

```

void Beep(uint32_t freq, uint32_t dur) {

    if (freq == 0) { TIM2->CCR3 = 0; return; }

    uint32_t period = 1000000 / freq;

    TIM2->ARR = period - 1;

    TIM2->CCR3 = period / 2;

    Delay_ms(dur);

    TIM2->CCR3 = 0;

}

```

```

void Music_Start(void) {

    Beep(1397, 100); Delay_ms(50);

    Beep(1760, 100); Delay_ms(50);

    Beep(2093, 300);

}

```

```

/* = LED = */

```

```

#define SR_LATCH_LO (GPIOA->ODR &= ~(1 << 4))

#define SR_LATCH_HI (GPIOA->ODR |= (1 << 4))

#define SR_SCK_LO   (GPIOA->ODR &= ~(1 << 5))

#define SR_SCK_HI   (GPIOA->ODR |= (1 << 5))

#define SR_DATA_LO  (GPIOA->ODR &= ~(1 << 7))

#define SR_DATA_HI  (GPIOA->ODR |= (1 << 7))

```

```

void LED_Init(void) {

    GPIOA->MODER &= ~((3<<(4*2)) | (3<<(5*2)) | (3<<(7*2)));

    GPIOA->MODER |= (1<<(4*2)) | (1<<(5*2)) | (1<<(7*2));

    GPIOA->OSPEEDR |= (2<<(4*2)) | (2<<(5*2)) | (2<<(7*2));

    SR_LATCH_LO; SR_SCK_LO;

}

```

```

void LED_Set(uint16_t data) {

    SR_LATCH_LO;

    for (int i = 15; i >= 0; i--) {

        SR_SCK_LO;

        if (data & (1 << i)) SR_DATA_HI;

        else SR_DATA_LO;

        for(int k=0; k<5; k++) __NOP();

        SR_SCK_HI;

        for(int k=0; k<5; k++) __NOP();

    }

    SR_LATCH_HI;

}

```

```

/* = LCD DISPLAY DRIVER = */

```

```

#define SDA_HI (GPIOB->ODR |= (1<<7))

#define SDA_LO (GPIOB->ODR &= ~(1<<7))

```



```

#define SCL_HI (GPIOB->ODR |= (1<<6))

#define SCL_LO (GPIOB->ODR &= ~(1<<6))

void I2C_Wait(void) { Delay_us(5); }

void I2C_Start(void) { SDA_HI; SCL_HI; I2C_Wait(); SDA_LO; I2C_Wait();
SCL_LO; I2C_Wait(); }

void I2C_Stop(void) { SDA_LO; SCL_LO; I2C_Wait(); SCL_HI; I2C_Wait();
SDA_HI; I2C_Wait(); }

void I2C_Send(uint8_t data) {

    for(int i=7; i>=0; i--) {

        if(data & (1<<i)) SDA_HI; else SDA_LO;

        I2C_Wait(); SCL_HI; I2C_Wait(); SCL_LO; I2C_Wait();

    }

    SDA_HI; I2C_Wait(); SCL_HI; I2C_Wait(); SCL_LO; I2C_Wait();

}

void PCF_Write(uint8_t data) {

    I2C_Start();

    I2C_Send(I2C_ADDR << 1);

    I2C_Send(data);

    I2C_Stop();

}

```

```

void LCD_Nibble(uint8_t nib, uint8_t rs) {

    uint8_t val = (nib & 0xF0) | (rs & 0x01) | 0x08;

    PCF_Write(val | 0x04);

    Delay_us(50);

    PCF_Write(val & ~0x04);

    Delay_us(50);

}

```

```

void LCD_Raw(uint8_t nib) {

    uint8_t val = (nib << 4) | 0x08;

    PCF_Write(val | 0x04);

    Delay_us(50);

    PCF_Write(val & ~0x04);

    Delay_us(50);

}

```

```

void LCD_Cmd(uint8_t cmd) {

    LCD_Nibble(cmd & 0xF0, 0);

    LCD_Nibble((cmd << 4) & 0xF0, 0);

    Delay_ms(2);

}

```

```

void LCD_Data(uint8_t data) {

    LCD_Nibble(data & 0xF0, 1);

}

```

```

    LCD_Nibble((data << 4) & 0xF0, 1);

    Delay_ms(1);

}

void LCD_String(char *str) { while (*str) LCD_Data(*str++); }

void LCD_Init(void) {

    GPIOB->MODER &= ~((3<<(6*2)) | (3<<(7*2)));

    GPIOB->MODER |= ((1<<(6*2)) | (1<<(7*2)));

    GPIOB->OTYPER |= (1<<6) | (1<<7);

    GPIOB->OSPEEDR |= (3<<(6*2)) | (3<<(7*2));

    GPIOB->ODR |= (1<<6) | (1<<7);

    Delay_ms(50);

    LCD_Raw(0x03); Delay_ms(5);

    LCD_Raw(0x03); Delay_ms(1);

    LCD_Raw(0x03); Delay_ms(1);

    LCD_Raw(0x02); Delay_ms(1);

    LCD_Cmd(0x28);

    LCD_Cmd(0x0C);

    LCD_Cmd(0x06);

    LCD_Cmd(0x01);

```

```

        Delay_ms(10);

    }

void LCD_Show(int sc, int tm, int lv) {

    LCD_Cmd(0x80);

    LCD_String("Score:"); LCD_Data((sc/10)+'0');
    LCD_Data((sc%10)+'0');

    LCD_String(" Lv:"); LCD_Data(lv+'0');


    LCD_Cmd(0xC0);

    LCD_String("Time:");

    LCD_Data((tm/10)+'0'); LCD_Data((tm%10)+'0');

    LCD_String("s ");

}


/* = INPUT BUTTON = */


volatile uint32_t LastInt = 0;


void Btn_Check(int id) {

    if ((Tick - LastInt) > 150) {

        Input = id;

        LastInt = Tick;

    }
}

```

```

}

void EXTI0_IRQHandler(void) { if(EXTI->PR & (1<<0)) { Btn_Check(0);
EXTI->PR |= (1<<0); } }

void EXTI1_IRQHandler(void) { if(EXTI->PR & (1<<1)) { Btn_Check(1);
EXTI->PR |= (1<<1); } }

void EXTI2_IRQHandler(void) { if(EXTI->PR & (1<<2)) { Btn_Check(2);
EXTI->PR |= (1<<2); } }

void EXTI3_IRQHandler(void) { if(EXTI->PR & (1<<3)) { Btn_Check(3);
EXTI->PR |= (1<<3); } }

void EXTI9_5_IRQHandler(void) { if(EXTI->PR & (1<<6)) { Btn_Check(4);
EXTI->PR |= (1<<6); } }

void EXTI15_10_IRQHandler(void) {

    if(EXTI->PR & (1<<12)) { Btn_Check(5); EXTI->PR |= (1<<12); }

    if(EXTI->PR & (1<<13)) { Btn_Check(6); EXTI->PR |= (1<<13); }

    if(EXTI->PR & (1<<14)) { Btn_Check(7); EXTI->PR |= (1<<14); }

    if(EXTI->PR & (1<<15)) { Btn_Check(8); EXTI->PR |= (1<<15); }

}

void Input_Init(void) {

    RCC->AHB1ENR |= (1 << 0) | (1 << 1);

    RCC->APB1ENR |= (1 << 0);

    RCC->APB2ENR |= (1 << 14);

    GPIOA->MODER &= ~(3 << (8*2));

    GPIOA->PUPDR |= (1 << (8*2));

```

```

        GPIOA->MODER &=
~( (3<<(0*2)) | (3<<(1*2)) | (3<<(2*2)) | (3<<(3*2)) | (3<<(6*2)) );

        GPIOA->PUPDR
|=  ( (1<<(0*2)) | (1<<(1*2)) | (1<<(2*2)) | (1<<(3*2)) | (1<<(6*2)) );

        GPIOB->MODER &=
~( (3<<(12*2)) | (3<<(13*2)) | (3<<(14*2)) | (3<<(15*2)) );

        GPIOB->PUPDR
|=  ( (1<<(12*2)) | (1<<(13*2)) | (1<<(14*2)) | (1<<(15*2)) );

        SYSCFG->EXTICR[0] &= ~0xFFFF;

        SYSCFG->EXTICR[1] &= ~(0xF << 8);

        SYSCFG->EXTICR[3] = 0x1111;

        EXTI->IMR |= 0xF04F;

        EXTI->FTSR |= 0xF04F;

        NVIC_EnableIRQ(EXTI0_IRQn); NVIC_EnableIRQ(EXTI1_IRQn);
        NVIC_EnableIRQ(EXTI2_IRQn); NVIC_EnableIRQ(EXTI3_IRQn);
        NVIC_EnableIRQ(EXTI9_5_IRQn);
        NVIC_EnableIRQ(EXTI15_10_IRQn);
    }

    int main(void) {

        SystemCoreClockUpdate();

        SysTick_Config(SystemCoreClock / 1000);

```

```

Input_Init();

LED_Init();

Audio_Init();

LCD_Init();


int Level = 1;


while (1) {

    LCD_Cmd(0x80); LCD_String("PRESS START!    ");

    LCD_Cmd(0xC0); LCD_String("TO PLAY GAME    ");


    while ((GPIOA->IDR & (1 << 8))) {

        Seed++;

        Delay_ms(10);

    }


    LCD_Cmd(0x01); LCD_String("READY...");

    Music_Start();


    int Score = 0;

    int TimeLeft = 60000;

    int LastLED = -1;

```

```

while (TimeLeft > 0) {

    Speed = 1300 - (Level * 100);

    LCD_Show(Score, TimeLeft / 1000, Level);

    int RandLED;

    do { RandLED = GetRandom() % 9; } while (RandLED ==
LastLED);

    LastLED = RandLED;

    LED_Set(1 << RandLED);

    Input = -1;

    int Hit = 0;

    int Spent = 0;

    int Step = 10;

    for (int t = 0; t < Speed; t += Step) {

        if (Input == RandLED) {

            Hit = 1; Score++;

            LED_Set(0);

            Beep(2000, 100);

            Spent = t;

            Input = -1;

```



```

        if (Score > 0 && Score % 10 == 0) {

            Level++;

            TimeLeft = 60000;

            Beep(3000, 200);

        }

        break;
    }

    else if (Input != -1 && Input != RandLED) {

        Beep(200, 300);

        if(Level > 1) Level--;

        Score = 0;

        TimeLeft = 60000;

        LCD_Cmd(0x01); LCD_String("WRONG INPUT!");

        Delay_ms(1000);

        Input = -1;

        break;

    }

    Delay_ms(Step);

}

if (!Hit && Input == -1) {

    Spent = Speed;

}

```

```

        TimeLeft -= Spent;

        LED_Set(0);

        Delay_ms(150); TimeLeft -= 150;
    }

    LCD_Cmd(0x01); LCD_String("GAME OVER!");

    LCD_Cmd(0xC0); LCD_String("Score: ");

    LCD_Data((Score/10)+'0'); LCD_Data((Score%10)+'0');

    Beep(1000, 500); Delay_ms(3000);
}

}

```

LAMPIRAN B: Datasheet Excerpts

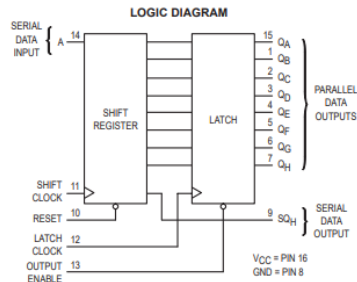
Alat	Spesifikasi						
STM32F411CEU6	STM32F411xC STM32F411xE Arm® Cortex®-M4 32b MCU+FPU, 125 DMIPS, 512KB Flash, 128KB RAM, USB OTG FS, 11 TIMs, 1 ADC, 13 comm. interfaces Datasheet - production data Features - Includes ST state-of-the-art patented technology - Dynamic efficiency line with BAM (Batch acquisition mode) 1.7 V to 3.6 V power supply -40°C to 85/105/125 °C temperature range - Core: Arm® 32-bit Cortex®-M4 CPU with FPU, adaptive real-time accelerator (ART Accelerator) allowing 0-wait state execution from flash memory, frequency up to 100 MHz, memory protection unit, 125 DMIPS/1.25 DMIPS/MHz (Dhrystone 2.1), and DSP instructions - Memories - Up to 512 Kbytes of flash memory 128 Kbytes of SRAM - Clock, reset, and supply management 1.7 V to 3.6 V application supply and I/Os - POR, PDR, PVD, and BOR 4-to-26 MHz crystal oscillator - Internal 16 MHz factory-trimmed RC 32 kHz oscillator for RTC with calibration - Internal 32 kHz RC with calibration - Power consumption - Run: 100 µA/MHz (peripheral off) - Stop (Flash in Stop mode, fast wakeup time): 42 µA typical at 25 °C; 65 µA max at 25 °C - Stop (Flash in Deep power down mode, slow wakeup time): down to 9 µA at 25 °C; 28 µA max at 25 °C - Standby: 1.8 µA at 25 °C / 1.7 V without RTC; 11 µA at 85 °C at 1.7 V - V_{BAT} supply for RTC: 1 µA at 25 °C 1×12-bit, 2.4 MSPS A/D converter: up to 16 channels - General-purpose DMA: 16-stream DMA controllers with FIFOs and burst support - Up to 11 timers: up to six 16-bit, two 32-bit timers up to 100 MHz, each with up to four IC/OC/PWM or pulse counter and quadrature (incremental) encoder input, two watchdog  timers (independent and window) and a SysTick timer - Debug mode - Serial wire debug (SWD) & JTAG interfaces - Cortex®-M4 Embedded Trace Macrocell™ - Up to 81 I/O ports with interrupt capability - Up to 78 fast I/Os up to 100 MHz - Up to 77 5 V-tolerant I/Os - Up to 13 communication interfaces - Up to 3 x I²C interfaces (SMBus/PMBus) - Up to 3 USARTs (2 x 12.5 Mbit/s, 1 x 6.25 Mbit/s), ISO 7816 interface, LIN, IrDA, modem control) - Up to 5 SPI/I²Ss (up to 50 Mbit/s, SPI, or I²S audio protocol), SPI2 and SPI3 with muxed full-duplex I²S to achieve audio class accuracy via internal audio PLL or external clock - SDIO interface (SD/MMC/eMMC) - Advanced connectivity: USB 2.0 full-speed device/host/OTG controller with on-chip PHY - CRC calculation unit 96-bit unique ID - RTC: subsecond accuracy, hardware calendar - All packages are ECOPACK2 compliant Table 1. Device summary <table><tr><th>Reference</th><th>Part number</th></tr><tr><td>STM32F411xC</td><td>STM32F411CC, STM32F411RC, STM32F411VC</td></tr><tr><td>STM32F411xE</td><td>STM32F411CE, STM32F411RE, STM32F411VE</td></tr></table> January 2024 DS10314 Rev 8 1/151 This is information on a product in full production. www.st.com	Reference	Part number	STM32F411xC	STM32F411CC, STM32F411RC, STM32F411VC	STM32F411xE	STM32F411CE, STM32F411RE, STM32F411VE
Reference	Part number						
STM32F411xC	STM32F411CC, STM32F411RC, STM32F411VC						
STM32F411xE	STM32F411CE, STM32F411RE, STM32F411VE						

MC74HC595A
**8-Bit Serial-Input/Serial or
Parallel-Output Shift
Register with Latched
3-State Outputs**
High-Performance Silicon-Gate CMOS

The MC74HC595A consists of an 8-bit shift register and an 8-bit D-type latch with three-state parallel outputs. The shift register accepts serial data and provides a serial output. The shift register also provides parallel data to the 8-bit latch. The shift register and latch have independent clock inputs. This device also has an asynchronous reset for the shift register.

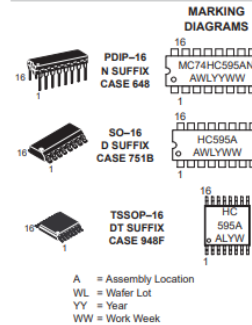
The HC595A directly interfaces with the SPI serial data port on CMOS MPUs and MCUs.

- Output Drive Capability: 15 LSTTL Loads
- Outputs Directly Interface to CMOS, NMOS, and TTL
- Operating Voltage Range: 2.0 to 6.0 V
- Low Input Current: 1.0 μ A
- High Noise Immunity Characteristic of CMOS Devices
- In Compliance with the Requirements Defined by JEDEC Standard No. 7A
- Chip Complexity: 328 FETs or 82 Equivalent Gates
- Improvements over HC595
 - Improved Propagation Delays
 - 50% Lower Quiescent Power
 - Improved Input Noise and Latchup Immunity



ON Semiconductor

<http://onsemi.com>

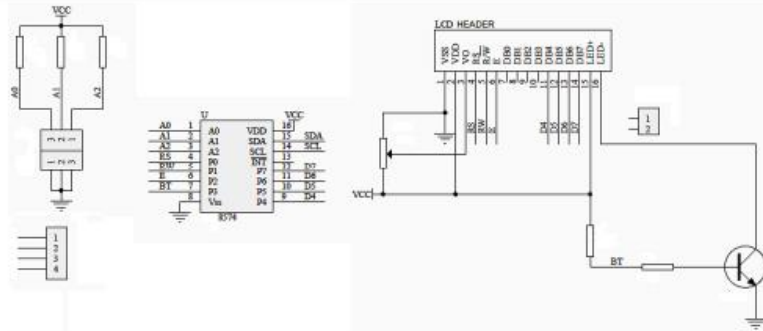


PIN ASSIGNMENT

Q _B	1	16	V _{CC}
Q _C	2	15	Q _A
Q _D	3	14	A
Q _E	4	13	OUTPUT ENABLE
Q _F	5	12	LATCH CLOCK
Q _G	6	11	SHIFT CLOCK
Q _H	7	10	RESET
GND	8	9	SO _H

ORDERING INFORMATION

Device	Package	Shipping
MC74HC595AN	PDIP-16	2000 / Box
MC74HC595AD	SOIC-16	48 / Rail
MC74HC595ADR2	SOIC-16	2500 / Reel
MC74HC595ADT	TSSOP-16	96 / Rail
MC74HC595ADTR2	TSSOP-16	2500 / Reel



Reference circuit diagram of the I2C-to-LCD piggy-back board.

I2C LCD Display.

At first you need to solder the I2C-to-LCD piggy-back board to the 16-pins LCD module. Ensure that the I2C-to-LCD piggy-back board pins are straight and fit in the LCD module, then solder in the first pin while keeping the I2C-to-LCD piggy-back board in the same plane with the LCD module. Once you have finished the soldering work, get four jumper wires and connect the LCD module to your Arduino as per the instruction given below.



LCD display to Arduino wiring.



Setting Up:

Hitachi's HD44780 based character LCD are very cheap and widely available, and is an essential part for any project that displays information. Using the LCD piggy-back board, desired data can be displayed on the LCD through the I2C bus. In principle, such backpacks are built around PCF8574 (from NXP) which is a general purpose bidirectional 8 bit I/O port expander that uses the I2C protocol. The PCF8574 is a silicon CMOS circuit provides general purpose remote I/O expansion (an 8-bit quasi-bidirectional) for most microcontroller families via the two-line bidirectional bus (I2C-bus). Note that most piggy-back modules are centered around PCF8574T (SO16 package of PCF8574 in DIP16 package) with a default slave address of 0x27. If your piggy-back board holds a PCF8574AT chip, then the default slave address will change to 0x3F. In short, if the piggy-back board is based on PCF8574T and the address connections (A0-A1-A2) are not bridged with solder it will have the slave address 0x27.



Address selection pads in the I2C-to-LCD piggy-back board.

Table 5. PCF8574A address map

Pin connectivity			Address of PCF8574A								Address byte value		7-bit hexadecimal address without R/W
A2	A1	A0	A6	A5	A4	A3	A2	A1	A0	R/W	Write	Read	
V _{SS}	V _{SS}	V _{SS}	0	1	1	1	0	0	0	-	70h	71h	38h
V _{SS}	V _{SS}	V _{DD}	0	1	1	1	0	0	1	-	72h	73h	39h
V _{SS}	V _{DD}	V _{SS}	0	1	1	1	0	1	0	-	74h	75h	3Ah
V _{SS}	V _{DD}	V _{DD}	0	1	1	1	0	1	1	-	76h	77h	3Bh
V _{DD}	V _{SS}	V _{SS}	0	1	1	1	1	0	0	-	78h	79h	3Ch
V _{DD}	V _{SS}	V _{DD}	0	1	1	1	1	0	1	-	7Ah	7Bh	3Dh
V _{DD}	V _{DD}	V _{SS}	0	1	1	1	1	1	0	-	7Ch	7Dh	3Eh
V _{DD}	V _{DD}	V _{DD}	0	1	1	1	1	1	1	-	7Eh	7Fh	3Fh

Address Setting of PCF8574A (extract from PCF8574A data specs).

Note: When the pad A0~A2 is open, the pin is pull up to VDD. When the pin is solder shorted, it is pull down to VSS.

The default setting of this module is A0~A2 all open, so is pull up to VDD. The address is 3Fh in this case.

Reference circuit diagram of an Arduino-compatible LCD backpack is shown below. What follows next is information on how to use one of these inexpensive backpacks to interface with a microcontroller in ways it was exactly intended.

LAMPIRAN C: Testing Results

Pengujian	Kondisi	Hasil	Status
Start button	Menekan tombol start	Masuk game, Score reset 0, Waktu set 60s	Berhasil
Input Benar	Menekan tombol sesuai LED nyala	LED mati, Score bertambah, Buzzer bunyi	Berhasil
Input Salah	Menekan tombol yang salah	Score reset 0, Level turun, Waktu reset	Berhasil
Level Up	Score mencapai kelipatan 10	Level naik, Speed bertambah, Waktu reset 60s	Berhasil
Game Over	Waktu habis 0 detik	Permainan selesai	Berhasil

LAMPIRAN D: Video Demonstration

https://drive.google.com/file/d/1a9I05yo_-Co9CFKtjSKuPB-SV0GFzeI6/view?usp=drive_link

CHECKLIST KELENGKAPAN LAPORAN

Content: 10%

- [☒] Abstrak
- [☒] BAB I-VII lengkap
- [☒] Minimum 20 halaman (tidak termasuk lampiran)
- [] Semua gambar, table diberi caption dan nomor

Technical: 40%

- [☒] Register configuration explained dengan tabel
- [☒] Schematic diagram, jelas dan lengkap
- [☒] Flowchart dan state machine included
- [☒] Photo dokumentasi hardware
- [☒] Video demonstration

CLO3 Requirements: 40 %

- [☒] Penerapan Serial communication (I2C/UART/SPI)
- [☒] Penerapan Timer configuration explained
- [☒] Interrupt system explained
- [☒] Register-level code documented
- [☒] Integration testing performed

Formatting: 10%

- [☒] Font: Times New Roman 12pt
- [☒] Line spacing: 1.5
- [☒] Margin: 4-3-3-3 cm
- [☒] Page numbers